



Instituto Infnet

O painel responde ao usuário

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 6 · Etapa 3 · Interação dinâmica

Prof. Renan Santos

Onde paramos (Aula 5)

O painel saiu do laptop: ambiente declarado, repositório, URL pública.

- **Ambiente reproduzível**

venv + requirements.txt + .gitignore — a receita, não a cópia.

- **Repositório e deploy**

GitHub → Community Cloud. git push = publicar.

- **Metade do TP2 entregue**

Falta o que faz alguém FICAR na página.

► Cole a URL do seu app no chat — vamos abrir algumas agora.



A resposta que ninguém previu

A gestora abriu o link... e voltou com mais três pedidos.

- **"Queria olhar só o Nordeste — e comparar com o Sudeste."**

O painel mostra exatamente o recorte que VOCÊ escolheu.

- **"Dá para tirar os estados muito pequenos?"**

Cada recorte novo exige que você mexa no código e publique de novo.

- **"Dá para mandar a tabela para a nossa analista?"**

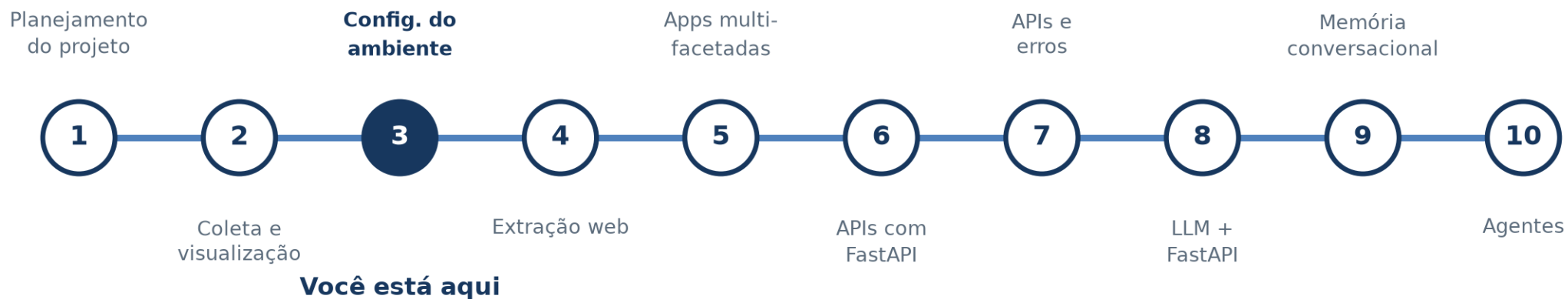
O link resolveu a distribuição — e criou um gargalo: VOCÊ.

Você virou o intermediário obrigatório de toda pergunta.

► A pergunta de hoje: como passar as decisões de recorte para quem usa o painel?

Onde estamos no Projeto de Bloco

Ainda na Etapa 3 — segunda metade: subcompetência 2.2, interação dinâmica.



Objetivos da aula

Cinco capacidades — e a primeira sustenta todas as outras.

- **Explicar o modelo de execução**

O que acontece de verdade quando alguém clica.

- **Usar o catálogo de widgets**

button, radio, checkbox, selectbox, multiselect, slider...

- **Agrupar entradas em st.form**

E saber quando o formulário é a escolha ERRADA.

- **Dar memória ao app**

st.session_state para o que atravessa interações.

- **Tratar fluxo e feedback**

st.stop, alertas, spinner, download — falhar com elegância.

No TP2

A Aula 5 entregou o link. Hoje entra o que faz alguém ficar mais de 10 segundos nele.

- **Já feito (Aula 5)**

Repositório + app publicado + ciclo de atualização.

- **Hoje**

Parâmetros nas mãos do usuário: filtros, ordenação, exportação.

- **Falta (Etapa 4)**

WebScraping, para fontes sem API — e o TP2 fecha.

► Um painel que só o autor consegue re-recortar não é um produto: é um relatório.

1 · A regra que explica tudo

O widget devolve um valor — não dispara um evento

Quem vem de JavaScript ou Tkinter procura um callback. Não existe.

- **regiao = st.selectbox('Região', [...])**
A variável regiao JÁ É a escolha do usuário.
Nenhuma função é registrada; nada é 'ouvido'.
- **Então como o app sabe que o usuário mexeu?**
Não sabe — e não precisa: ele simplesmente RODA TUDO DE NOVO.
- **Os dois fatos [Richards, p87]**
 - 1) Nada é guardado entre reruns, por padrão.
 - 2) A cada entrada do usuário, o script roda de cima para baixo.

► **Interação = o arquivo inteiro reexecutado com o novo valor.**

O ciclo, visto por inteiro

Uma regra, três consequências — e elas são o roteiro do resto da aula.

A regra que explica TUDO no Streamlit: cada interação roda o script inteiro

app.py

executado de cima para baixo, sempre inteiro

```
import streamlit as st

dados = obter_dados()    # @st.cache_data

regiao = st.selectbox("Região", ...)
minimo = st.slider("População mín.", ...)

filtrados = [l for l in dados if ...]
st.bar_chart(filtrados, ...)
```

1 • o usuário mexe num widget

2 • o Streamlit reexecuta o app.py INTEIRO

3 • a tela é redesenhada com o novo valor

Consequências:

variável comum é ZERADA a cada rerun

coleta cara → @st.cache_data

guardar algo → st.session_state

e de novo, a cada clique

O widget não 'dispara um evento': ele DEVOLVE UM VALOR, e o script inteiro roda de novo com esse valor.

A prova: o contador que nunca passa de 1

passo03_rerun.py

```
contador = 0                                # nasce zerado em TODO rerun
if st.button("Somar 1"):
    contador += 1
st.metric("Valor", contador)

# Clique 5 vezes. O valor continua 1.
# A linha 'contador = 0' roda ANTES do if, sempre.
```

O que sobrevive a um rerun

A tabela que resolve 90% das dúvidas de Streamlit.

- **SOBREVIVE: `st.session_state`**

Dicionário global que persiste durante a sessão do usuário [Richards, p91].

- **SOBREVIVE: resultado de `@st.cache_data`**

É o que você já usa desde a Aula 2 — agora com explicação estrutural.

- **SOBREVIVE: o valor do widget**

Ligado à chave dele; o Streamlit o devolve de novo.

- **NÃO SOBREVIVE: variável comum**

Qualquer coisa recalculada no topo do script é refeita do zero.

► Em Streamlit, variável comum NÃO é memória. Memória precisa ser pedida.

Uma regra, quatro consequências

Tudo o que vamos usar hoje sai de uma única regra.

- **Variável comum é zerada a cada rerun**
→ `st.session_state` (daqui a pouco)
- **Coleta cara seria refeita a cada rerun**
→ `@st.cache_data` (já feito na Aula 2)
- **4 filtros mexidos = 4 reexecuções**
→ `st.form` (daqui a pouco)
- **Não faz sentido desenhar a tela agora**
→ `st.stop()` (daqui a pouco)

► **Decore a regra, não a lista de funções: as funções são consequência.**

ATIVIDADE · O que este código faz? (A7)

- `total = 0 ; if st.button('Somar 1'): total += 1 ; st.metric('Total', total)`
- O usuário clica TRÊS vezes. O que aparece em Total?
- A) 3 B) 1 C) 0 D) depende da velocidade dos cliques
- Vote (30s) → convença o par (2-3 min) → revote → rodamos ao vivo

 8 min



2 · O catálogo de widgets

Os cinco do capítulo 5

Cada um devolve um tipo diferente — e é isso que decide como você o usa.

- **st.button → True só no rerun do clique**

Ação pontual ('Favoritar'). [Raghavendra, p114]

 No rerun seguinte volta a False — a mensagem 'some'.

- **st.radio / st.selectbox → UMA opção**

Critério de ordenação; escolher a UF. [Raghavendra, p115, p119]

- **st.checkbox → True/False**

Mostrar ou esconder a tabela. [Raghavendra, p117]

- **st.multiselect → uma LISTA**

Filtrar regiões. Você já usou na Aula 2. [Raghavendra, p121]

► O tipo de retorno é o contrato: lista pede comprehension, bool pede if.

Widgets aplicados ao Painel ODS

app.py

```
regioes = st.multiselect("Regiões", disponiveis, default=disponiveis)
criterio = st.radio("Ordenar por", list(CRITERIOS))
mostrar = st.checkbox("Mostrar a tabela", value=True)

# a lógica não muda: continua Python puro, sem pandas
filtrados = ordenar(filtrar(dados, regioes, minimo), criterio)
```

st.slider — e uma nota sobre a fonte

O capítulo chama-se 'Buttons and Sliders'... e não traz um único exemplo de slider.

- **O que o livro entrega no cap. 5**

Botão, rádio, checkbox, drop-down, multiselect, download, progresso, spinner.

O resumo do próprio autor (p127) NÃO menciona sliders.

- **De onde vem o que ensinamos aqui**

Documentação oficial do Streamlit, verificada no demo desta aula.

- **Por que isso é conteúdo, e não fofoca**

Checar a fonte — inclusive o livro adotado — faz parte do trabalho.

O mesmo hábito vale para toda resposta de IA.

► **Título de capítulo não é evidência. Rodar é.**

Slider — e a armadilha do `min == max`

app.py

```
minimo = st.slider("População mínima", min_value=pop_min,  
                  max_value=pop_max, value=pop_min, step=100_000)
```

```
# 🐛 se pop_min == pop_max o st.slider LEVANTA EXCEÇÃO  
# acontece quando os limites vêm dos dados e o filtro esvazia a base
```

```
def faixa_populacao(linhas):          # src/transformacoes.py  
    ...                               # garante largura mínima 1
```

Anti-padrões de interatividade

Os erros clássicos ao dar controles ao usuário.

- **Painel de avião**

12 controles porque 'podiam ser úteis'. Transfira as decisões que IMPORTAM.

- **Guardar estado em variável comum**

Não guarda nada — é a regra do bloco anterior.

- **Estado decorativo**

Guardar uma lista e não fazer nada com ela.

Aconteceu neste demo: um 'Favoritar' que só imprimia siglas.

- **Controle sem tratamento do caso vazio**

Gráfico vazio, métricas zeradas, ou exceção.

▶ Cada widget é uma decisão que você transfere — e uma responsabilidade que você assume.

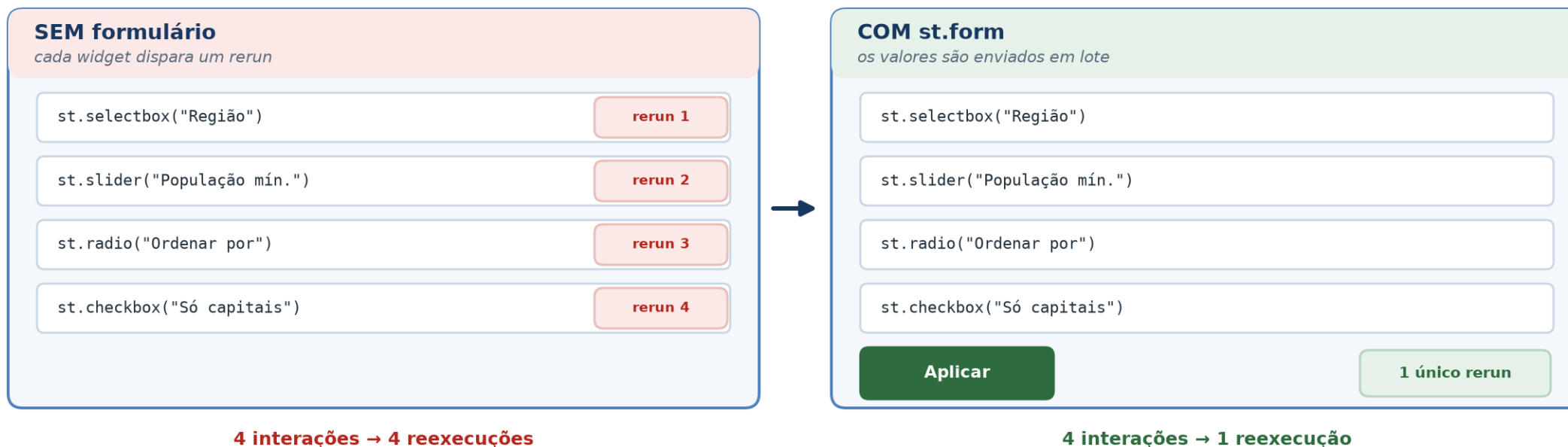


3 · Formulário, memória e fluxo

Quatro mexidas, quantos reruns?

'the application needs to rerun (...) causing a bad user experience' [Raghavendra, p157].

Widgets soltos × formulário: quantas vezes o script roda?



st.form agrupa; st.form_submit_button envia. Sem o submit dentro do form, o Streamlit acusa erro.

st.form: enviar em lote

app.py

```
with st.form("filtros"):
    regioes = st.multiselect("Regiões", disponiveis, default=disponiveis)
    minimo = st.slider("População mínima", pop_min, pop_max, pop_min)
    criterio = st.radio("Ordenar por", list(CRITERIOS))
    mostrar = st.checkbox("Mostrar a tabela", value=True)
    aplicar = st.form_submit_button("Aplicar filtros")
```

A lista que esquece — e como resolver

Favoritar SP, depois RJ, e sobra só ['RJ']: é o exemplo do livro [Richards, p87–90].

Por que a lista 'esquece' — e como `st.session_state` resolve

SEM `session_state`

a variável nasce de novo a cada rerun

```
favoritas = []          # rerun -> []  
if st.button("Favoritar"):  
    favoritas.append(uf)
```

clique 1 → rerun ["SP"]

clique 2 → rerun ["RJ"] <- SP sumiu

clique 3 → rerun ["MG"] <- RJ sumiu



COM `session_state`

dicionário global que sobrevive aos reruns

```
if "favoritas" not in st.session_state:  
    st.session_state.favoritas = []  
if st.button("Favoritar"):  
    st.session_state.favoritas.append(uf)
```

clique 1 → rerun ["SP"]

clique 2 → rerun ["SP", "RJ"]

clique 3 → rerun ["SP", "RJ", "MG"]

A memória dura a SESSÃO: recarregar a página começa tudo de novo. É memória do usuário, não banco de dados.

st.session_state: dar memória ao app

app.py

```
# 1. INICIALIZAR – só na primeira execução da sessão
if "favoritas" not in st.session_state:
    st.session_state.favoritas = []

# 2. LER e ESCREVER – como num dicionário comum
if st.button("Comparar"):
    st.session_state.favoritas.append(uf)

# 3. e a memória tem de SERVIR para alguma coisa:
comparadas = comparar(dados, st.session_state.favoritas)
```

Estado precisa de uma RAZÃO de estado

A primeira versão deste demo guardava as UFs marcadas... e não fazia nada com elas.

- **O que havia antes**

Um botão 'Favoritar' que imprimia: Favoritas: SP · RJ · MG.

Código certo, recurso inútil — o 'painel de avião' desta mesma aula.

- **Por que isso é pior do que parece**

session_state decorativo DESTRÓI o próprio argumento.

Se some ao mudar o filtro e ninguém sente falta, variável comum bastava.

- **O que o comparador faz agora**

% da população do Brasil · posição no ranking nacional · distância para a maior marcada.

E lê a base COMPLETA: marque SP e AC, filtre só o Sudeste — o AC continua lá.

► **Teste antes de criar estado: o que o usuário perde se isso sumir a cada rerun?**

APIs do livro que envelheceram

O livro-texto é de 2023. Cinco funções mudaram de nome — use a coluna da direita.

- **@st.cache → @st.cache_data / @st.cache_resource**

Dados × conexões e modelos [Richards, p83].

- **@st.experimental_memo → @st.cache_data**

@st.experimental_singleton → @st.cache_resource

- **st.experimental_rerun() → st.rerun()**

st.experimental_show() → st.write() para depurar [Richards, p80].

- **A lição maior**

Livro, tutorial e resposta de IA envelhecem.

Rode, leia o aviso de descontinuação, confira na documentação.

► Peça à IA qual versão ela está assumindo — metade dos exemplos usa a API velha.

st.stop: responder bem ao caso vazio

app.py

```
if not filtrados:
    st.warning("Nenhuma UF atende a esses filtros. Diminua a "
              "população mínima ou selecione mais regiões.")
    st.stop()      # nada abaixo é executado

# sem isto: gráfico vazio, métricas zeradas – uma tela sem sentido
```

ATIVIDADE · Conserte o comparador (A8)

- O trecho no slide tem TRÊS defeitos. Em duplas:
- 1) ache os três e escreva o SINTOMA que o usuário veria
- 2) reescreva corrigido
- 3) marque qual QUEBRA o app e qual apenas o faz ENGANAR

 10 min

E agora, ao vivo: publicar a v4

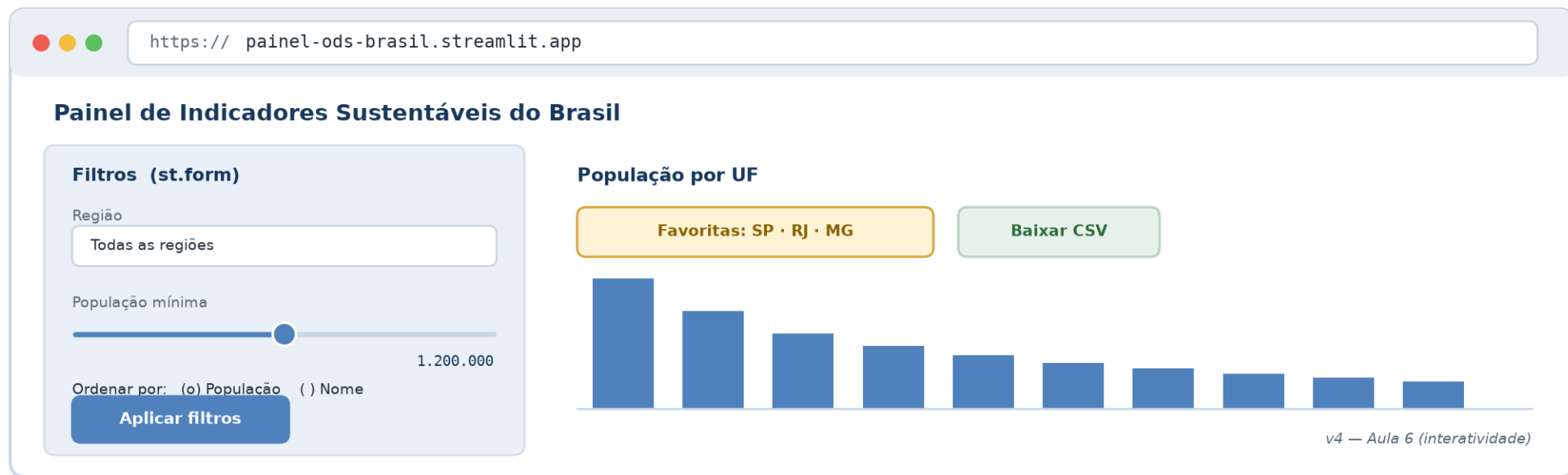
terminal

```
git add .  
git commit -m "v4: filtros em formulário, favoritas e download"  
git push  
  
# recarregue a URL da Aula 5 em 1-2 minutos  
# ambiente (a5) -> repositório (a5) -> deploy (a5) -> iteração (hoje)
```

O mesmo link, agora explorável

Nenhuma linha de coleta mudou. Mudou quem decide o recorte.

O resultado da Aula 6: o mesmo link, agora explorável



Mesma URL, mesmo repositório — um `git push` depois.

Princípios que levamos

Quatro ideias que valem para qualquer interface de dados.

- **O widget é uma expressão, não um evento**

Entendido o rerun, o resto do Streamlit é consequência.

- **Estado explícito — e justificado**

O que atravessa interações vai para `st.session_state`, num lugar só.
Se você não sabe dizer o que se perde sem ele, não é estado: é enfeite.

- **Degradar com elegância**

Todo controle exige resposta para TODAS as escolhas, inclusive as vazias.

- **A interface expõe parâmetros; a lógica fica isolada**

dados → lógica → interface.

► **Interatividade é responsabilidade: quem oferece o controle responde pela escolha.**



O caso, em 3 lições (v4)

O que construir a v4 de verdade ensinou.

- **O widget não é um evento: é um valor**

A partir daí, nada no Streamlit precisa ser decorado.

- **Memória precisa ser pedida**

session_state para o que atravessa; cache para o que é caro; variável para o resto.

- **Interatividade é responsabilidade**

O defeito que ENGANA é pior que o que quebra — ninguém o descobre.

E todo controle tem de responder: para que ele serve?

► Um app que mente é mais perigoso que um app que falha.

ATIVIDADE · Bilhete de saída

- Qual parâmetro do SEU painel faz mais sentido entregar ao usuário — e por quê?
- Ele deveria estar num formulário ou solto?

 **3 min**